```
import Foundation

/*:
 # 📘 MARK: Property — конспект (на примере Марка)

 **Оглавление:** 🖥️ Код · 📻 String · 🔢 Int · 📦 Структура · 🏠 Хранимое
  · 🎒 Дефолт vs init · 🌡️ Computed · 🔮 Наблюдатели · 🧬 Value Types · 🚪
  mutating · 🏴‍☠️ Инженерный фокус · 📊 Итог · 🚀 Запуск
 */

// MARK: - Property
// TS: user's profile

struct UserProfile {
    let name: String = "Vale" // 1️⃣ Stored Properties — ОС айфона физически
     выделяет 16 байт, значение лежит там const, пока жив весь объект
     UserProfile

    var followersCount: Int = 12 { // 2️⃣ stored properties с наблюдателем
     didSet
        didSet {
            print("didSet: \(oldValue) → \(followersCount)")
        }
    } // [Исправлено: закрыли свойство здесь, чтобы код ниже работал]

    var counterText: String { // 3️⃣ computed property — «вычисляемое» в IT
     не только математика, НО процесс динамического создания значения в
     момент запроса
        return "Followers: \(followersCount)."
    }

    mutating func addNewFollower() {
        followersCount += 1 // Вот тут мы физически прибавили единицу в
         память!
        print("Hooray! New subscriber!")
    }
}

// 🚀 Запуск (перенесён из тела свойства наружу, чтобы шпаргалка была
 рабочей):
var profile = UserProfile()
print(profile.counterText) // Followers: 12.

profile.addNewFollower() // Hooray! New subscriber!
print(profile.counterText) // Followers: 13.

// MARK: - 📻 String — 16 байт

/*:
 ## 📻 String — 16 байт (Полный текст)
```

Тип String в Swift — это продвинутая структура. Сама строка внутри себя делит эти 16 байт (ровно 2 машинных слова по 8 байт) на хранение служебной информации: длину строки, флаги и указатель на адрес в памяти.

Сколько бы букв ни было в имени — 4 буквы ("Vale") или «Война и мир» на миллион символов — само свойство name внутри твоей структуры UserProfile всегда будет занимать ровно 16 байт.

Stack: Это быстрая память для структуры UserProfile. Когда ты создаешь var profile = UserProfile(), вся структура целиком ложится в Stack. Свойство name забирает там свои 16 байт.

Heap: Это большая общая память устройства. Если текст короткий (до 15 символов), Swift применять оптимизацию Small String Optimization и умудряется закодировать буквы прямо внутри этих 16 байт Stack, вообще не трогая Heap. Но если текст длинный, Swift выделяет под него отдельное место в Heap, а в своих 16 байтах в Stack просто хранит «ссылку-адрес» (указатель), ведущую к этому тексту.

Почему let name, а var followersCount? Имя — let, потому что не меняется: Марк всегда Марк. followersCount — var, потому что меняется (addNewFollower). Если let followersCount — ошибка в addNewFollower (cannot assign to let).

💡 Лайфхак: почему 16 байт — всегда

Чтобы не думать «а сколько байт займёт моя строка?», Swift фиксирует размер: 16 байт — всегда. Хоть "Vale" (4 буквы), хоть «Война и мир» (миллион символов).

Как это работает:

- Короткий текст (≤15 символов) — Swift прячет буквы внутри этих 16 байт. Heap не трогает. 🏠 Hab: груз внутри корпуса.
- Длинный текст — Swift кладёт его в Heap, а в 16 байтах держит указатель (адрес). 🏠 Hab: груз снаружи, в корпусе — координаты.

Что важно понять: name в UserProfile — 16 байт, независимо от длины имени. Это не «16 + длина текста». 16 — всегда.
*/

```
print(MemoryLayout<String>.size)   // 16
print(MemoryLayout<String?>.size)  // 16

// MARK: - 🔢 Int — 8 байт

/*:
 ## 🔢 Int — 8 байт (Полный текст)

 • Свойство типа Int на всех современных Айфонах занимает ровно 8 байт (64 бита).
```

• В отличие от строк, числа — ребята простые. Никаких скрытых указателей или Heap им не нужно. Цифра 12 физически записывается прямо в эти 8 байт в Stack в виде единиц и нулей (0000...1100).

💡 Лайфхак: почему Int — честный парень

Чтобы не путаться с указателями, запомни: Int — самый простой тип. 8 байт — и всё внутри. Никаких Heap, никаких ссылок.

Как это работает:

• Число 12 → процессор пишет прямо в 8 байт Stack: 00000000...00001100.
• Хочешь прочитать — процессор берёт эти 8 байт. Всё.
• Heap не участвует. 🥫 паёк: 8 см еды, всё внутри, никаких ссылок.

Что важно понять: Int всегда 8 байт. Int? — 9 (8 + пломба). Но сам Int — 8, без сюрпризов.
*/

```
print(MemoryLayout<Int>.size)    // 8
print(MemoryLayout<Int?>.size)   // 9

// MARK: - 📦 Вся структура — 24 байта

/*:
 ## 📦 Вся структура — 24 байта (Полный текст)

 Вся твоя структура UserProfile в Stack изначально весит всего 24 байта (16 байт на строку + 8 байт на число). Очень компактно!
 */

print(MemoryLayout<UserProfile>.size)   // 24

// MARK: - 🏠 Хранимое свойство

/*:
 ## 🏠 Что такое хранимое свойство? (Полный текст)

 Хранимое свойство — это то, у чего есть ячейка в памяти структуры.

 String (16 байт), Int (8 байт), Bool (1 байт) — все базовые типы, объявленные через var/let, — это хранимые свойства.

 Именно они дают 24 байта у UserProfile: 16 (name) + 8 (followersCount) = 24.

 А counterText — не хранимое (ячейки нет) — это вычисляемое (блок 🌡️).

 **Образ:** 🏠 стеллажи в Hab. Каждый — физическое место (ячейка). name — 16 см полки. followersCount — 8 см полки. Вместе 24 см. counterText — не полка, а надпись на стене (блок 🌡️).
```

```
*/

// MARK: - 🎒 Дефолт vs init

/*:
## 🎒 Дефолт в коде vs Инициализатор (init) (Полный текст)

**Пометка:** это отдельная тема — про инициализацию, не про память. К computed (0 байт) вернёмся в 🌡️. Контраст «24 vs 0» — не здесь, а в 📦 → 🌡️.

### 🏭 Конвейер vs 🎨 Кастомизация

Дефолт — это конвейер: все объекты сходят с завода одинаковые. init — это кастомизация: каждый объект настраивается отдельно под ситуацию.

### 🧠 Главный инсайт

- В ПАМЯТИ (ЖЕЛЕЗО): Разницы НЕТ. Процессор Apec-3 выделяет одинаковые ячейки RAM. Объект весит байт-в-байт одинаково.
- В ЛОГИКЕ (КОД): Дефолтное значение — это то, что запрограммировано в NASA изначально для всех (стандарт). Инициализатор (init) — это то, что Марк Уотни настраивает вручную под свои экстремальные условия.

Почему UserProfile не нужен init, а Hab — нужен? У UserProfile оба свойства с дефолтом ("Vale", 12) → UserProfile() сам работает. У MartianHab — crewMemberName без дефолта → нужен init, чтобы сказать, кто живёт в Hab.

### 👩‍🚀 Код Жилого Модуля (Хаба) Apec-3 (Полный текст)

У NASA есть структура для жилых модулей. В ней хранимые свойства: waterSupply (запас воды), crewMemberName (кто внутри) и isEvacuated (эвакуирована ли миссия).

```swift
struct MartianHab {
    // 1️⃣ ДЕФОЛТНОЕ ЗНАЧЕНИЕ (Стандарт NASA для всех миссий)
    // По умолчанию Хаб полностью готов к работе, эвакуация отключена
    var isEvacuated: Bool = false

    var crewMemberName: String
    var waterSupply: Int {
        didSet { print("didSet: \\(oldValue) → \\(waterSupply)") }
    }
    var counterText: String {
        "Water: \\(waterSupply)L"
    }

    // 2️⃣ ИНИЦИАЛИЗАТОР (Создание конкретных условий)
```
```

```
    // Мы не можем запустить симуляцию Хаба, пока не укажем, кто там сидит
     и сколько у него воды
    init(crewMemberName: String, waterSupply: Int) {
        self.crewMemberName = crewMemberName
        self.waterSupply = waterSupply
        // isEvacuated сюда не пишем — оно уже по дефолту false
    }

    mutating func addNewWaterSource() {
        waterSupply += 1
        print("Hooray! New water source!")
    }
 }
```

Зачем self.?

self.crewMemberName — это свойство структуры. crewMemberName (без self) — параметр init. Они одноимённы — self различает: «свойство» vs «параметр».

🛠️ Ситуации на Марсе (Полный текст)

🛠️ Ситуация А: старт миссии (всё по плану)

В системе создается объект для Марка Уотни. Воды у него изначально много, статус эвакуации — дефолтный false.

```
swift
var hab = MartianHab(crewMemberName: "Марк Уотни", waterSupply: 250)
// Что в памяти компьютера Арес-3?
// crewMemberName = "Марк Уотни"
// waterSupply    = 250
// isEvacuated    = false (взялось дефолтное значение)
```

🛠️ Ситуация Б: налетает песчаная буря, экипаж улетает, Марк остаётся один

Происходит авария. Экипаж эвакуируется, думая, что Марк погиб. Но Марк выжил! Ему нужно изменить статус Хаба и жёстко пересчитать воду.

Так как свойства хранимые, Марк может залезть в память компьютера и перезаписать эти ячейки:

```
swift
hab.isEvacuated = true   // Меняем дефолтное false на true (миссия
 официально эвакуирована)
hab.waterSupply = 50     // Часть баков повреждена, перезаписываем старое
 число на 50

// Что теперь лежит в тех же ячейках памяти?
// crewMemberName = "Марк Уотни"
// isEvacuated    = true (перезаписано!)
// waterSupply    = 50  (перезаписано!)
```

Работает, потому что hab — var, и свойства — var. Если let — ❌ нельзя (блок 🌡️).

💡 Лайфхак Уотни: Инициализатор с дефолтом внутри

```
Чтобы компьютеру NASA не делать двойную работу (сначала создать Hab с
 дефолтными значениями, а потом сразу переписывать их в init), в Swift
 можно задавать дефолты прямо в аргументах init: init(crewMemberName:
 String, waterSupply: Int, isEvacuated: Bool = false).
*/
/*:
 ### 💡 Лайфхак Уотни (продолжение): Код WatneyHab

 ```swift
 struct WatneyHab {
     var crewMemberName: String
     var waterSupply: Int // В литрах

     // Задаем дефолт прямо в параметры конструктора.
     // Если Марк не укажет воду — NASA подставит стандартные 250 л.
     init(crewMemberName: String, waterSupply: Int = 250) {
         self.crewMemberName = crewMemberName
         self.waterSupply = waterSupply
     }
 }

 // Способ 1: Для обычного дня.
 // Марк не указывает воду → срабатывает дефолт из init → в ячейку пишется
  250.
 let normalDay = WatneyHab(crewMemberName: "Марк Уотни")
 // waterSupply = 250

 // Способ 2: Для дня, когда Марк собирается сжигать водород, чтобы
  получить воду вручную.
 // Марк указывает 600 → дефолт НЕ срабатывает → в ячейку пишется 600.
 let customDay = WatneyHab(crewMemberName: "Марк Уотни", waterSupply: 600)
 // waterSupply = 600
Что важно понять: дефолт в параметре init — это не дефолт свойства. Здесь у
 свойства waterSupply нет = 250. Дефолт живёт в сигнатуре init(waterSupply:
 Int = 250). Не передал — подставился. Передал — взялось твоё.

Образ: 🎒 рюкзак у входа в Hab. NASA положила туда 250 л — «по умолчанию».
 Марк не лезет в рюкзак → берёт 250. Марк знает, что ему нужно 600 → кладёт
 своё, рюкзак не трогает.
*/

// Рабочий WatneyHab
struct WatneyHab {
var crewMemberName: String
var waterSupply: Int
init(crewMemberName: String, waterSupply: Int = 250) {
self.crewMemberName = crewMemberName
self.waterSupply = waterSupply
}
}
let normalDay = WatneyHab(crewMemberName: "Марк Уотни")
print(normalDay.waterSupply) // 250
let customDay = WatneyHab(crewMemberName: "Марк Уотни", waterSupply: 600)
print(customDay.waterSupply) // 600
```
```

```
// Рабочий MartianHab (для Ситуаций А и Б)
struct MartianHab {
var isEvacuated: Bool = false
var crewMemberName: String
var waterSupply: Int {
didSet { print("didSet: (oldValue) → (waterSupply)") }
}
var counterText: String { "Water: (waterSupply)L" }

init(crewMemberName: String, waterSupply: Int) {
self.crewMemberName = crewMemberName
self.waterSupply = waterSupply
}

mutating func addNewWaterSource() {
waterSupply += 1
print("Hooray! New water source!")
}
}

// Ситуация А
var hab = MartianHab(crewMemberName: "Марк Уотни", waterSupply: 250)
print(hab.crewMemberName, hab.waterSupply, hab.isEvacuated)

// Ситуация Б
hab.isEvacuated = true
hab.waterSupply = 50
print(hab.crewMemberName, hab.waterSupply, hab.isEvacuated)

/*:

📊 Итоговая сводка по свойствам (Память и Логика)
Свойство / Элемент конспекта    Размер в памяти    Где физически находится в RAM    Логическая суть (Образ)
String name    16 байт    Stack (до 15 симв.) / Указатель ведёт в Heap (если длинный текст)    Фиксированная ячейка под текст («Война и мир» или "Vale" — весит одинаково).
Int followersCount    8 байт    Строго в Stack    Честный паёк. Число пишется битами напрямую.
Computed (counterText)    0 байт    Нигде (не имеет ячейки)    Инструкция мелом на стене. Вычисляется заново при каждом запросе.
Дефолтное значение    Столько же, сколько тип    Выделяется стандартная RAM    Заводской конвейер NASA. Одинаково для всех объектов на старте.
Инициализатор (init)    Столько же, сколько тип    Выделяется стандартная RAM    Кастомизация. Каждый объект — свой.
⚠️ Почему в нашем коде «нельзя»?
swift
MartianHab(crewMemberName: "Марк", waterSupply: 250, isEvacuated: true)  // ❌ ошибка
Причина — кастомный init «съедает» memberwise init. Не дефолт запрещает.
```

Без кастомного init — Swift сам генерирует memberwise, дефолт можно переопределить:

swift
struct Hab { var isEvacuated: Bool = false; var name: String }
var h = Hab(isEvacuated: true, name: "Марк") // ✅ можно!
Вывод: в памяти — одинаково. Разница только в логике: дефолт = «стандарт для всех», init = «настроил под конкретный объект».
*/

// MARK: - 🌡️ Computed property — 0 байт

/*:

🌡️ Computed property — 0 байт
Вычисляемое свойство (counterText) — НЕ ЗАНИМАЕТ ни одного байта в памяти структуры! Для него нет ячейки. В памяти устройства строки "Followers: 12." не существует. Computed property — это, на самом деле, маскировка для функции (метода). Процессор в этот самый миг «просыпается» и выполняет блок кода внутри фигурных скобок:

Он бежит в ячейку памяти, где хранится followersCount.

Вытаскивает оттуда текущую цифру (например, 12).

Берет шаблон "Followers: ".

Склеивает их вместе, выделяет под этот результат временную память на долю секунды, отдает её в print и тут же уничтожает строку.

Если ты вызовешь print(profile.counterText) десять раз подряд, процессор 10 раз заново выполнит это склеивание. Stored — это готовая вещь, которая лежит на полке в памяти. Computed — это инструкция для процессора собрать вещь заново в тот момент, когда мы её попросили.

💡 Лайфхак: 0 байт, но не бесплатно
Чтобы не думать «computed — это даром?», запомни: памяти — 0, но времени — каждый раз.

Как это работает:

- counterText не хранится. Нет ячейки. 0 байт.
- Каждый print(profile.counterText) → процессор заново: бежит в followersCount (8 байт) → берёт число (12) → склеивает "Followers: 12." → отдаёт, уничтожает.

Что важно понять: 10 вызовов = 10 расчётов. Stored — готовая вещь на полке. Computed — инструкция собрать заново. 🌡️ датчик на пульте: показания не хранятся, считываются в момент запроса.
*/

// MARK: - 🦙 Наблюдатель didSet

/*:

🐕 Наблюдатель didSet
didSet не занимает памяти. Это код, не ячейка. followersCount всё ещё 8 байт, didSet — 0.

Наблюдатель didSet — это сторожевой пес для твоей переменной. Как только в эти 8 байт памяти записывается любое новое число, процессор на долю секунды ставит программу на паузу и выполняет код внутри didSet.

Посадив 'пса' didSet прямо на свойство followersCount, ты гарантируешь: кто бы и из какого бы угла программы ни изменил это число в памяти, триггер сработает ВСЕГДА и на 100% стабильно. Изменение памяти и реакция на него теперь намертво связаны в одном месте.

Причём «любое новое число» — это буквально любое присваивание. Даже если ты напишешь followersCount = 12, когда там уже 12, — пёс всё равно проснётся и гавкнет. Swift не сравнивает old с new, ему достаточно факта записи.

Примеры с Марком
1. Срабатывает после записи — внутри уже новое значение

swift
var waterLiters = 100 {
    didSet { print("💧 В баке: \\(waterLiters) л (было: \\(oldValue))") }
}
waterLiters = 50 // 💧 В баке: 50 л (было: 100)
Что происходит в железе по шагам:

Марк открывает кран и сливает воду. Говорит процессору: «Запиши в waterLiters число 50».

Процессор идёт к ячейке waterLiters (8 байт в Stack). Там лежит 100.

Сначала процессор запоминает старое число 100 в oldValue — на долю секунды, чтобы было с чем сравнить.

Потом процессор перезаписывает 100 → 50. Ячейка уже содержит 50.

Только теперь просыпается didSet. Смотрит в ячейку — а там уже 50. oldValue = 100.

didSet печатает: «В баке: 50 л (было: 100)».

Логика: didSet — после записи. Внутри — новое значение. Если бы он срабатывал до — внутри было бы 100. Но нет — 50.

2. Срабатывает на факт записи — даже если значение то же

swift
var oxygenPercent = 100 {
    didSet { print("🫁 Датчик кислорода: \\(oxygenPercent)%") }

```
}
oxygenPercent = 100 // 🫁 Датчик кислорода: 100%
```

Что происходит в железе по шагам:

Марк смотрит на датчик. Там 100%. Он решает ничего не менять — но всё равно нажимает кнопку «записать 100».

Процессор идёт к ячейке oxygenPercent. Там уже 100.

Процессор запоминает 100 в oldValue.

Процессор перезаписывает 100 → 100. Формально запись была — даже если число то же.

didSet просыпается — потому что была запись. Swift не проверяет «а изменилось ли?». Ему достаточно факта: «кто-то тронул ячейку».

didSet печатает: «Датчик кислорода: 100%».

Логика: didSet — сторож. Он реагирует на каждое открытие двери. Даже если тот же человек вернулся — сторож всё равно гавкнет. Swift не говорит: «А, это тот же — можно не лаять». Лает всегда.

Где это важно: если didSet обновляет что-то дорогое (например, пересчитывает график или шлёт запрос на сервер) — ты потратишь ресурсы впустую, если присвоишь то же значение. Проверяй сам: if newValue != oldValue { ... }.

3. Не срабатывает при init — только при внешнем присваивании

```
swift
struct Hab {
    var potatoesKg: Int {
        didSet { print("🥔 Картошки стало: \\(potatoesKg) кг") }
    }
    init(potatoesKg: Int) {
        self.potatoesKg = potatoesKg   // ← НЕ выстрелит
    }
}
var hab = Hab(potatoesKg: 15)   // тишина — init
hab.potatoesKg = 12             // 🥔 Картошки стало: 12 кг — внешнее
```

Что происходит в железе по шагам:

Создание (Hab(potatoesKg: 15)):

Марк собирает новый Hab. Он говорит: «Картошки — 15 кг».

Процессор выделяет ячейку под potatoesKg (8 байт в Stack).

init записывает 15 в ячейку. Это первая запись — рождение объекта.

didSet молчит. Почему? Потому что didSet не сторожит рождение. Он сторожит изменения после рождения. Если бы didSet срабатывал при init — он бы печатал «Картошки стало: 15» при каждом создании Hab. Это шум, не польза.

Тишина. Объект создан, didSet не тронут.

Внешнее присваивание (hab.potatoesKg = 12):

Марк съел 3 кг. Он говорит процессору: «Запиши 12».

Процессор идёт к той же ячейке (объект уже существует). Там 15.

Процессор запоминает 15 в oldValue. Перезаписывает → 12.

didSet просыпается — потому что это внешнее присваивание, не init.

Печатает: «Картошки стало: 12 кг».

Логика: didSet — сторож после рождения. При рождении (init) — не сторожит. Иначе каждый новый Hab орал бы при создании. Тишина при рождении — правильно.

4. willSet vs didSet — до и после

swift
```
var rationPacks = 6 {
    willSet { print("📦 Было \\(rationPacks), станет \\(newValue)") }
    didSet  { print("📦 Было \\(oldValue), стало \\(rationPacks)") }
}
rationPacks = 5
// 📦 Было 6, станет 5   ← willSet
// 📦 Было 6, стало 5    ← didSet
```
Что происходит в железе по шагам:

Марк берёт один паёк. Он говорит процессору: «Запиши в rationPacks число 5».

Процессор видит: у ячейки два сторожа — willSet и didSet.

Сначала просыпается willSet. Он срабатывает ДО записи. В ячейке ещё 6. Но willSet уже знает, что будет 5 — это newValue. Печатает: «Было 6, станет 5».

willSet закрывается. Процессор запоминает 6 в oldValue (для didSet).

Процессор перезаписывает 6 → 5. Теперь в ячейке 5.

Теперь просыпается didSet. Он срабатывает ПОСЛЕ записи. В ячейке уже 5. oldValue = 6. Печатает: «Было 6, стало 5».

Логика:

- willSet — до записи. В ячейке старое значение. Но есть newValue — «что будет».

```
• didSet — после записи. В ячейке новое. Есть oldValue — «что было».
• Оба срабатывают на любое присваивание. Порядок: willSet → запись → didSet.

Когда что использовать:

• willSet — если надо подготовиться к изменению (например, сохранить старое
 значение куда-то).
• didSet — если надо отреагировать на изменение (например, обновить UI или
 записать в лог).
• В 90% случаев используют didSet. willSet — реже.
*/

// Рабочие примеры didSet
var waterLiters = 100 {
didSet { print("💧 В баке: (waterLiters) л (было: (oldValue))") }
}
waterLiters = 50

var oxygenPercent = 100 {
didSet { print("🫁 Датчик кислорода: (oxygenPercent)%") }
}
oxygenPercent = 100

struct Hab {
var potatoesKg: Int {
didSet { print("🥔 Картошки стало: (potatoesKg) кг") }
}
init(potatoesKg: Int) { self.potatoesKg = potatoesKg }
}
var habPotato = Hab(potatoesKg: 15)
habPotato.potatoesKg = 12

var rationPacks = 6 {
willSet { print("📦 Было (rationPacks), станет (newValue)") }
didSet { print("📦 Было (oldValue), стало (rationPacks)") }
}
rationPacks = 5

// MARK: - 🧬 Структуры — Value Types

/*:

🧬 Структуры — Value Types
Структуры в Swift — это типы значений (Value Types). структуры д.б. max
 защищенными, монолитными и неизменяемыми.

По умолчанию Swift запрещает любым функциям внутри структуры менять её
 собственные stored properties.
*/

// MARK: - 🚪 mutating
```

/*:

🚪 mutating
Ключевое слово mutating (от слова mutation — мутация, изменение) — это твое официальное разрешение для Swift: «Внимание! Эта функция собирается перетряхнуть память самой структуры!».

mutating — только для struct. Для class не нужен (метод меняет объект по ссылке) и запрещён: 'mutating' isn't valid on methods in classes.

💡 Лайфхак: почему без mutating — нельзя
Чтобы не путаться «зачем это слово?», запомни: struct по умолчанию заморожен. mutating — ключ, который разрешает писать в свои байты.

Как это работает:

- func addNewFollower() без mutating → компилятор: «Ты меняешь followersCount, но не сказал mutating. Запрещаю». Ошибка: 'self' is immutable.
- mutating func addNewFollower() → компилятор: «Понял, меняешь копию. Разрешаю».

Что важно понять: mutating — не «пересоздание». Это разрешение на запись в ту же ячейку Stack. 🚪 шлюз: без шлюза не выйдешь из Hab, с шлюзом — выходишь и меняешь что нужно.
*/

// **MARK: - 📓 Инженерный фокус**

/*:

📓 Инженерный фокус: что происходит в памяти?
Когда ты вызываешь profile.addNewFollower(), в железе айфона происходит именно замена одной цифры в ячейке памяти — прямо на месте, без копирования.

mutating — это не «пересоздание» структуры. Это разрешение компилятору менять self. Процессор просто записывает новое число 13 в те же самые 8 байт в Stack, где лежало 12. Никакой новой структуры, никакого уничтожения старой.

Copy-On-Write — это другая история, к mutating отношения не имеет. COW срабатывает, когда value-тип с ссылкой на Heap (Array, String, Dictionary, Set) шарится между двумя переменными:

swift
var a = [1, 2, 3]
var b = a // пока НЕ копируется — обе смотрят на один буфер
b.append(4) // ← вот ТУТ COW: буфер копируется, потому что он общий
В твоём UserProfile COW не участвует вообще — там только String + Int, никакого шаринга.

Именно поэтому мы обязаны создавать объект профиля через var profile. Если написать let profile, компилятор не даст вызвать mutating-метод — ошибка ещё до запуска: cannot use mutating member on immutable value. Это проверка на уровне языка, а не «процессор физически не смог бы». Процессор бы смог — но Swift не разрешает.

💡 Лайфхак: почему b = a — не копия
Чтобы не думать «а вдруг массив скопируется?», запомни: сначала — нет. Копия — только при записи.

Как это работает:

- var b = a → копируется только указатель (8 байт). Буфер общий. 🧬 близнецы: одна квартира.
- b.append(4) → Swift видит: буфер общий → копирует его → теперь у b свой. 🧬 близнецы: начал ремонт — вторая квартира отдельная.

Что важно понять: COW = ленивое копирование. «Не копируй, пока не попросят». Пока читаешь — экономия. Как только пишешь — копия. 📓 бортжурнал: пока читают — один, как только пишут — копия.
*/

```
// Демо COW
var a = [1, 2, 3]
var b = a
b.append(4)
print("a: (a), b: (b)")
```

// MARK: - 📊 Итог

/*:

📊 Итог по структуре UserProfile
- let name — выделили 16 байт в Stack, заморозили навсегда.
- var followersCount — выделили 8 байт в Stack под число.
- didSet — караулит эти 8 байт. Как только они меняются, берет old value в oldValue и запускает триггер.
- var counterText — памяти не занимает вообще, работает как функция «на лету» в момент вызова.
- mutating func — дает процессору разрешение менять ячейки этой структуры на месте (не пересоздавать).
*/

// MARK: - 🚀 Запуск механизма в работу

/*:

🚀 Запуск механизма в работу
Именно здесь написанная тобой структура («чертеж») превращается в реальный живой объект в памяти айфона.

🚀.1 var profile = UserProfile()

- Что делает процессор: Смотрит на чертеж UserProfile и понимает, что для создания объекта ему нужно 24 байта (16 байт на имя + 8 байт на число). Он мгновенно находит эти свободные 24 байта в памяти Stack и выделяет их под переменную profile.
- Что внутри: В этот момент внутри ячеек памяти прописываются дефолтные значения: имя становится "Vale", а счетчик — 12.
- Почему var: Мы пишем var, потому что в будущем планируем вызывать mutating func addNewFollower(). Эта функция меняет ячейки структуры на месте. Если написать let profile, компилятор запретит вызов mutating-метода — ошибка ещё до запуска.

🚀.2 print(profile.counterText)

- Что делает процессор: Он видит запрос к computed property counterText. Помнишь? Памяти под него нет!
- Процесс: Процессор «на лету» прыгает в фигурные скобки этого свойства, заглядывает в соседние 8 байт (где лежит число 12), склеивает строку "Followers: 12." и выводит её на экран в консоль. Строка тут же исчезает из памяти.

🚀.3 profile.addNewFollower()

- Что делает процессор: Запускает функцию «мутации».
- Цепочка событий внутри железа:

Выполняется строчка followersCount += 1. Число 12 в памяти меняется на 13.

В этот же миг «просыпается» сторожевой пес didSet. Он подхватывает old value 12 в переменную oldValue и печатает твой первый принт: didSet: 12 → 13.

Блок didSet закрывается. Процессор возвращается в функцию addNewFollower() и выполняет следующую строчку — печатает второй принт: Hooray! New subscriber!.

🚀.4 print(profile.counterText) — снова

- Что делает процессор: Мы снова просим его выдать текст. Он опять выполняет код computed property с нуля. Но теперь, заглянув в ячейку памяти счетчика, он видит там уже не 12, а свежее число 13. Строка собирается заново, и на экран выводится актуальный результат: Followers: 13.

*/

```
print("\n--- 🚀 Финальный запуск ---")
var profile2 = UserProfile()
print(profile2.counterText)
profile2.addNewFollower()
print(profile2.counterText)
```